```
    tft.setCursor(endButtonX - 10, endButtonY - 7); // Center text inside the circle
    tft.print("End");
  }

  bool isCallButtonTouched(int x, int y) {
    int buttonRadius = 30;
    int buttonX = 80;
    int buttonY = 180; // Same as callButtonY in drawButtons

    int dx = x - buttonX;
    int dy = y - buttonY;
    return (dx * dx + dy * dy <= buttonRadius * buttonRadius);
    call.answer();
  }

  bool isEndButtonTouched(int x, int y) {
    int buttonRadius = 30;
    int buttonX = isCallActive ? 120 : 160; // Center for second screen
    int buttonY = 180; // Same as endButtonY in drawButtons

    int dx = x - buttonX;
    int dy = y - buttonY;
    return (dx * dx + dy * dy <= buttonRadius * buttonRadius);
    call.hangoff();
  }

  void startCall() {
    isCallActive = true;
    callStartTime = millis();
```

Fig. 4: Code snippet for UI verification

Fig. 5: Indusphone dial pad UI

idle screen code, which checks for incoming calls, network connectivity, and battery levels.

Fig. 4 presents the code snippet for verifying UI input and incorporating functionality for call attendance and hang-up based on touch input. Similarly, the UI for dialling numbers and making calls is seamlessly integrated. Fig. 5 showcases the Indusphone dial pad UI.

Fig. 6 illustrates the MIC and speaker connection with the GSM module. Once the function for attending or hanging up a call is initiated, the GSM module independently manages the receiver. If a number is dialled or a call is attended, the sound transmits through the small-sized SMD speaker

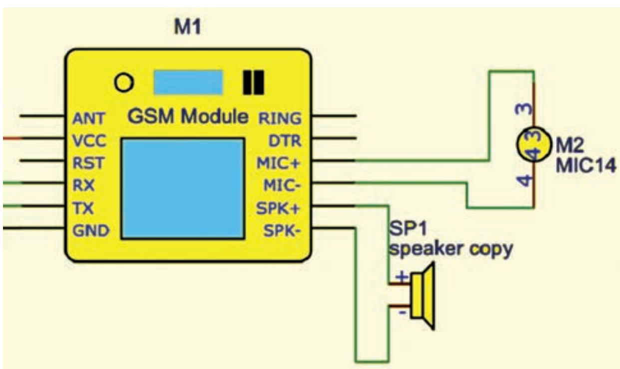

Fig. 6: MIC and speaker connection with GSM module